ALTKOM

Aplikacje kontenerowe w klastrach z wykorzystaniem Docker oraz Kubernetes

Dzień 2 · Kubernetes: od poda do aplikacji pod własnym adresem

Dawid Marniok

9–10 listopada 2026 · Altkom Akademia

Powtórka z dnia 1: trzy pytania

- czym różni się obraz od kontenera?
- dlaczego dane bazy trzymamy w wolumenie?
- co robi kontroler w Kubernetesie, gdy pod znika?

kubectl get nodes → 3 węzły Ready? To jedziemy.

Plan dnia 2

Start	Powtórka, sprawdzenie klastra	
Temat 1	Pod i Namespace → lab06	LAB
Temat 2	Deployment i konfiguracja (część 1)	
	Przerwa	
Temat 2	ConfigMap i Secret → lab07	LAB
Temat 3	Service i Ingress → lab08	LAB
	Przerwa obiadowa	
Temat 4	Ćwiczenie końcowe: ToDos na klastrze → lab09	LAB
	Przerwa	
Temat 5	Co dalej: dane, kontrolery, autoskalowanie, Helm, produkcja	
Koniec	Podsumowanie szkolenia, pytania, ankieta	

Dzisiejsza mapa: osiem obiektów Kubernetesa

Każdy rozwiązuje jeden problem — razem dają działającą aplikację

Pod	Namespace	ReplicaSet	Deployment	ConfigMap	Secret	Service	Ingress

Pod	uruchamia kontener(y)
Namespace	dzieli klaster na przestrzenie
ReplicaSet	pilnuje liczby podów
Deployment	wdraża nowe wersje
ConfigMap · Secret	konfiguracja i hasła z zewnątrz
Service	stały adres dla podów
Ingress	adres dla przeglądarki

1

DZIEŃ 2 · TEMAT 1

Pod i Namespace

Po tym temacie tworzysz pod z pliku YAML, dostajesz się do niego i diagnozujesz pody, które nie wstają.

1
Pod i Namespace

2
Deployment i konfiguracja

3
Service i Ingress

4
ToDos na klastrze

5
Co dalej

Pod: najmniejsza jednostka — nie kontener



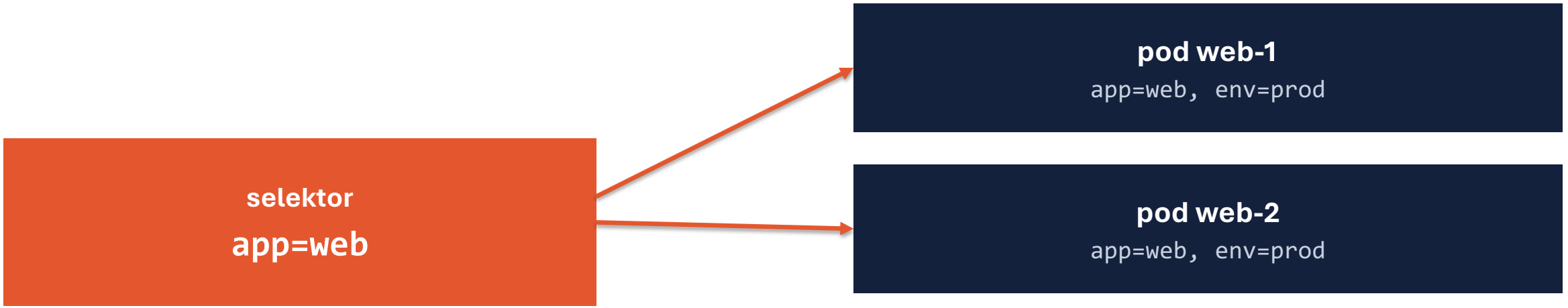
- zwykle jeden kontener na pod
- pod ma własny adres IP w klastrze
- pod jest jednorazowy: nie naprawia się, tylko zastępuje nowym
- nowy pod = nowe IP i nowa nazwa
- gołych podów prawie nie tworzymy — robi to Deployment

Każdy obiekt ma te same cztery części

```yaml
apiVersion: v1          # wersja API dla tego typu
kind: Pod               # typ obiektu
metadata:               # kto to jest
  name: web
  labels:
    app: web
spec:                   # czego chcę
  containers:
    - name: nginx
      image: nginx:1.29
      ports:
        - containerPort: 80
      resources:
        requests: { cpu: 50m, memory: 64Mi }
        limits:   { cpu: 200m, memory: 128Mi }
```

- apiVersion + kind: co to za obiekt
- metadata: nazwa, etykiety
- spec: stan pożądany — piszesz Ty
- status: stan faktyczny — pisze klaster
- kubectl explain pod.spec — wszystkie pola

Etykiety i selektory: tak obiekty znajdują się nawzajem




```
kubectl get pods -l app=web
kubectl get pods --show-labels
```

Service, Deployment i ReplicaSet wybierają pody po etykietach, nie po nazwach

Namespace: przestrzenie w jednym klastrze

Do czego

- oddzielenie zespołów i środowisk: dev, test
- nazwy unikalne tylko w namespace
- uprawnienia (RBAC) i limity zasobów per namespace

Czego NIE daje

- izolacji sieciowej — pody widzą się między namespace'ami
- osobnych węzłów — wszystko na tych samych maszynach
- bezpieczeństwa samego z siebie

W praktyce

- kubectl get pods -n kube-system
- kubectl get pods -A
- u nas domyślny: szkolenie

Status poda mówi, gdzie szukać problemu

STATUS	Co znaczy	Sprawdź
Pending	nie ma węzła z wolnymi zasobami albo brakuje wolumenu	describe → Events
ContainerCreating	pobieranie obrazu, montowanie wolumenów	chwilę poczekaj, potem describe
ImagePullBackOff	obrazu nie da się pobrać: literówka, tag, prywatny rejestr	describe → Events
CrashLoopBackOff	kontener startuje i pada, Kubernetes ponawia coraz rzadziej	logs (gdy pusto: --previous)
Running, READY 0/1	działa, ale nie przechodzi readinessProbe	describe, logs
OOMKilled	przekroczył limit pamięci	describe → Last State

Diagnostyka zawsze w tej samej kolejności

kubectl get pods
JAKI status?

→

kubectl describe pod
Events: CO się stało?

→

kubectl logs
(--previous)
CO powiedziała aplikacja?

```
Events:
  Type     Reason   From     Message
  Warning  Failed   kubelet  Failed to pull image "redis:9.9": not found
  Warning  BackOff  kubelet  Back-off pulling image "redis:9.9"
```

Sekcja Events na dole describe odpowiada na 80% pytań „czemu to nie działa”

DEMO

Pod, który nie wstaje

Tworzę pod z YAML, wchodzę do środka, potem trzy awarie i ich diagnoza

```
cd demo/d2-01-pod-diagnostyka
kubectl apply -f pod.yaml
kubectl get pods -o wide
kubectl port-forward pod/www 8086:80
kubectl exec -it www -- bash

kubectl apply -f awarie.yaml
kubectl get pods
kubectl describe pod awaria-pamiec
kubectl describe pod awaria-wezel
kubectl describe pod awaria-sonda
```

OBSERWUJCIE

- IP poda i węzeł w get -o wide
- OOMKilled, Pending i Running 0/1 — trzy różne statusy
- przyczyna zawsze w describe: Last State albo Events
- Running nie znaczy gotowy — patrzcie na READY

LAB `labs/lab06-pod/`

Pody i diagnostyka

1. pod z pliku YAML, port-forward, exec, logi
2. etykiety: zmiana przez apply i wybieranie -l
3. trzy zepsute pody: znajdź przyczynę każdego
4. pod z dwoma kontenerami: wspólne localhost

SUKCES: przyczyny trzech awarii zapisane na czacie, zepsuty-1 naprawiony

2

DZIEŃ 2 · TEMAT 2

Deployment i konfiguracja

Po tym temacie wdrażasz aplikację, która sama się naprawia, aktualizujesz ją bez przerwy i podajesz konfigurację z zewnątrz.

1 Pod i Namespace

2 Deployment i konfiguracja

3 Service i Ingress

4 ToDos na klastrze

5 Co dalej

Problem: goły pod nie wraca

Pod | Namespace | ReplicaSet | Deployment | ConfigMap | Secret | Service | Ingress

```
kubectl delete pod web
kubectl get pods
No resources found
```

- pod padł albo węzeł się wyłączył — koniec
- nikt nie pilnuje liczby kopii
- nowa wersja = ręcznie usuń i utwórz (przerwa)

Deployment: „chcę 3 kopie tej wersji” — i Kubernetes tego pilnuje

Deployment → ReplicaSet → Pody



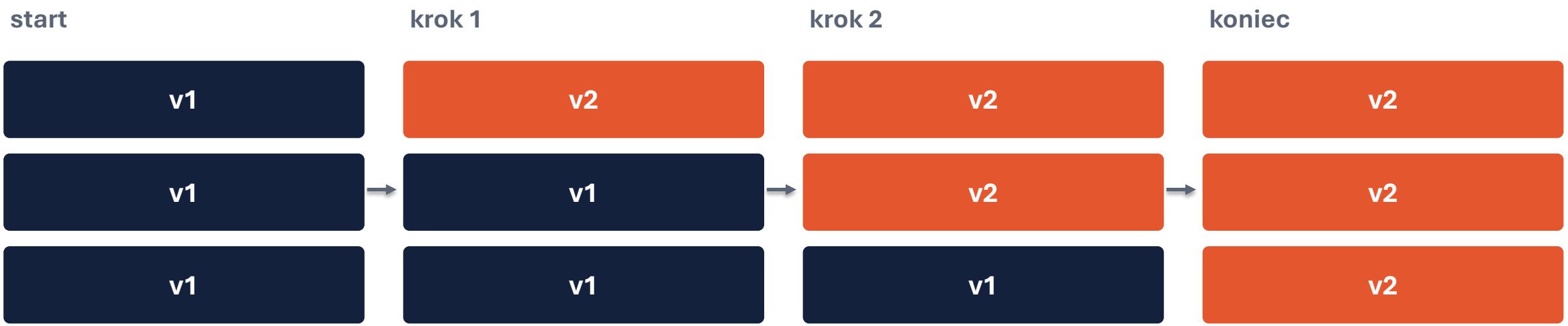
- Deployment: którą wersję chcesz i jak ją wymieniać
- ReplicaSet: pilnuje dokładnej liczby podów jednej wersji — tworzysz go pośrednio
- nazwa poda = nazwa Deploymentu + hash ReplicaSetu + losowy sufiks

Deployment w YAML: pod w środku, jako szablon

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello
spec:
  replicas: 3
  selector:
    matchLabels:
      app: hello              # które pody są moje
  template:                   # szablon poda
    metadata:
      labels:
        app: hello            # musi pasować do selektora
    spec:
      containers:
        - name: hello
          image: nginxdemos/hello:0.3-plain-text
```

- replicas: ile kopii
- selector = etykiety z template
- template to zwykły pod bez nazwy
- zmiana w template = nowa wersja (rollout)
- zmiana replicas = tylko skalowanie

Rolling update: wymiana podów po kolei, bez przerwy




```
# deployment.yaml: image: nginxdemos/hello:0.4-plain-text
kubectl apply -f deployment.yaml
kubectl rollout status deploy/hello
kubectl rollout undo deploy/hello      # awaryjnie cofnij
```

- nowy pod musi być gotowy, zanim stary zniknie
- zła wersja? stare pody nadal obsługują ruch

Bez readinessProbe „gotowy” = „proces wystartował”. To za mało.

Sondy: skąd Kubernetes wie, że aplikacja działa

Sonda	Pytanie	Gdy nie przechodzi
readinessProbe	czy mogę przyjmować ruch?	pod wypada z Service (bez restartu)
livenessProbe	czy żyję, czy się zawiesiłem?	kontener restartowany
startupProbe	czy skończyłem się uruchamiać?	wstrzymuje pozostałe sondy (wolny start)

Liveness sprawdza tylko samą aplikację. Zależności (baza) — w readiness.

Requests i limits: rezerwacja i sufit

requests	limits
• ile scheduler rezerwuje na węźle • decyduje, na który węzeł trafi pod • cpu: 100m = 0,1 rdzenia • memory: 128Mi	• maksimum, którego pod nie przekroczy • CPU ponad limit → spowolnienie • pamięć ponad limit → OOMKilled • restart kontenera

Zawsze ustawiaj requests. Limit pamięci = ochrona węzła przed wyciekiem pamięci.

ConfigMap i Secret: konfiguracja poza obrazem

Pod | Namespace | ReplicaSet | Deployment | **ConfigMap** | **Secret** | Service | Ingress

```yaml
kind: ConfigMap
metadata:
  name: api-config
data:
  APP_VERSION: "1.0"
---
kind: Secret
metadata:
  name: api-secret
stringData:
  DB_PASSWORD: changeme
---
# w Deploymencie, przy kontenerze:
envFrom:
  - configMapRef:
      name: api-config
  - secretRef:
      name: api-secret
```

- jeden obraz, różne środowiska: dev, test, prod
- jako zmienne środowiskowe (envFrom, env)
- albo jako pliki zamontowane w kontenerze
- zmiana zmiennych wymaga restartu podów

Pułapka: Secret to nie szyfrowanie

Co jest w środku

- data: hasło zakodowane base64
- echo Y2hhbmdlbWU= | base64 -d → changeme
- każdy, kto może czytać Secrety, zna hasła
- Secret w YAML w Git = hasło w Git

Jak to robić

- RBAC: kto może czytać Secrety
- szyfrowanie etcd po stronie klastra
- zewnętrzny sejf: Key Vault, Secrets Manager, Vault
- External Secrets Operator, Sealed Secrets, SOPS

DEMO

Rolling update, rollback i konfiguracja

Wymieniam wersję na żywo, psuję ją i cofam, potem podaję konfigurację

```
cd demo/d2-02-rollout-konfiguracja
kubectl apply -f deployment.yaml
kubectl get pods -w            # drugi terminal

# w pliku: image: traefik/whoami:v1.11.0
kubectl apply -f deployment.yaml
kubectl rollout status deploy/whoami

# w pliku: image: traefik/whoami:v9.9
kubectl apply -f deployment.yaml
kubectl rollout undo deploy/whoami

kubectl apply -f configmap.yaml
```

OBSERWUJCIE

- pody v2 pojawiają się, zanim znikną v1
- zła wersja: stare pody dalej Running
- rollout undo przywraca stary ReplicaSet
- zmienne z ConfigMap dopiero po restarcie

LAB

`labs/lab07-deployment-konfiguracja/`

Deployment, aktualizacja i konfiguracja

1. Deployment: usuń pod i zobacz, jak wraca
2. skalowanie: kubectl scale kontra replicas w pliku
3. rolling update, zła wersja i rollout undo
4. ConfigMap i Secret jako zmienne środowiskowe

SUKCES: po rollout undo działa poprzednia wersja, a env pokazuje KOLOR i DB_PASSWORD

3

DZIEŃ 2 · TEMAT 3

Service i Ingress

Po tym temacie Twoja aplikacja ma stały adres w klastrze i adres, który otworzysz w przeglądarce.

1	2	3	4	5
Pod i Namespace	Deployment i konfiguracja	**Service i Ingress**	ToDos na klastrze	Co dalej

Service: stały adres dla podów, które się zmieniają

Pod | Namespace | ReplicaSet | Deployment | ConfigMap | Secret | **Service** | Ingress



- adres IP i nazwa Service się nie zmieniają · listę podów aktualizuje klaster (EndpointSlice)
- ruch trafia tylko do podów gotowych (readinessProbe)

Typy Service: kto ma się dostać do aplikacji

Typ	Dostępny z	Kiedy
ClusterIP	tylko z wnętrza klastra	domyślny: komunikacja między usługami (api, baza)
NodePort	port 30000–32767 na każdym węźle	testy, środowiska bez load balancera
LoadBalancer	zewnętrzny IP od chmury	wystawienie jednej usługi na świat (płatne)
Ingress (+ ClusterIP)	jeden adres, wiele usług po nazwie hosta	aplikacje HTTP — najczęściej

DNS w klastrze: nazwa Service to adres

```
# z poda w tym samym namespace
wget -qO- http://hello

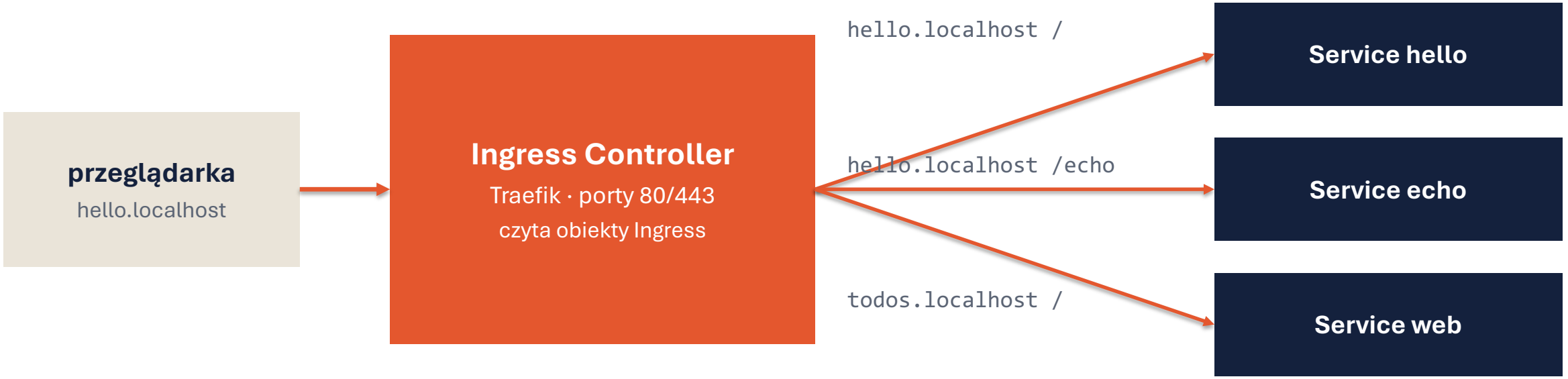
# z innego namespace
wget -qO- http://hello.szkolenie

# pełna nazwa
hello.szkolenie.svc.cluster.local

# ToDos: web → api → mongo
API_URL=http://api:3000
MONGO_URL=mongodb://mongo:27017
```

- CoreDNS w kube-system
- <service>.<namespace>.svc.cluster.local
- w tym samym namespace wystarczy <service>
- dlatego Service w ToDos musi nazywać się api

Ingress: jeden wjazd, routing po nazwie hosta i ścieżce




```
rules:
  - host: hello.localhost
    http: { paths: [ { path: /, pathType: Prefix, backend: { service: { name: hello, port: { number: 80 } } } } ] }
```

Ingress dziś, Gateway API jutro

Ingress

- stabilny, prosty, wszędzie wspierany
- host + ścieżka → Service
- reszta (TLS, przepisywanie, limity) w adnotacjach kontrolera
- na tym szkoleniu: Ingress

Gateway API

- następca Ingressu, standard Kubernetesa
- podział ról: infrastruktura (Gateway) i aplikacja (HTTPRoute)
- nagłówki, wagi ruchu (canary), TCP/gRPC bez adnotacji
- warto znać przy nowych projektach

DEMO

Service i Ingress

Trzy pody, jeden stały adres w klastrze — i adres dla przeglądarki

```
cd demo/d2-03-service-ingress
kubectl apply -f app.yaml
kubectl get pods -l app=kto -o wide
kubectl get endpointslices \
  -l kubernetes.io/service-name=kto

kubectl run klient --rm -it --image=busybox:1.37 -- sh
  wget -qO- http://kto | grep Hostname
  nslookup kto

kubectl scale deploy kto --replicas=1
kubectl apply -f ingress.yaml
curl http://kto.localhost
```

OBSERWUJCIE

- trzy IP podów, jeden CLUSTER-IP Service
- nazwa kto działa jak w Compose — to DNS klastra
- scale zmienia listę endpointów, adres zostaje
- X-Forwarded-Host: żądanie przeszło przez Traefika

LAB `labs/lab08-service-ingress/`

Service i Ingress

1. Service ClusterIP i EndpointSlice: adresy podów
2. test z wnętrza klastra: wget http://hello
3. zepsuty selektor — pusta lista endpointów
4. Ingress: http://hello.localhost i ścieżka /echo

SUKCES: curl http://hello.localhost odpowiada różnymi podami, a /echo — drugą aplikacją

4

DZIEŃ 2 · TEMAT 4

ToDos na klastrze

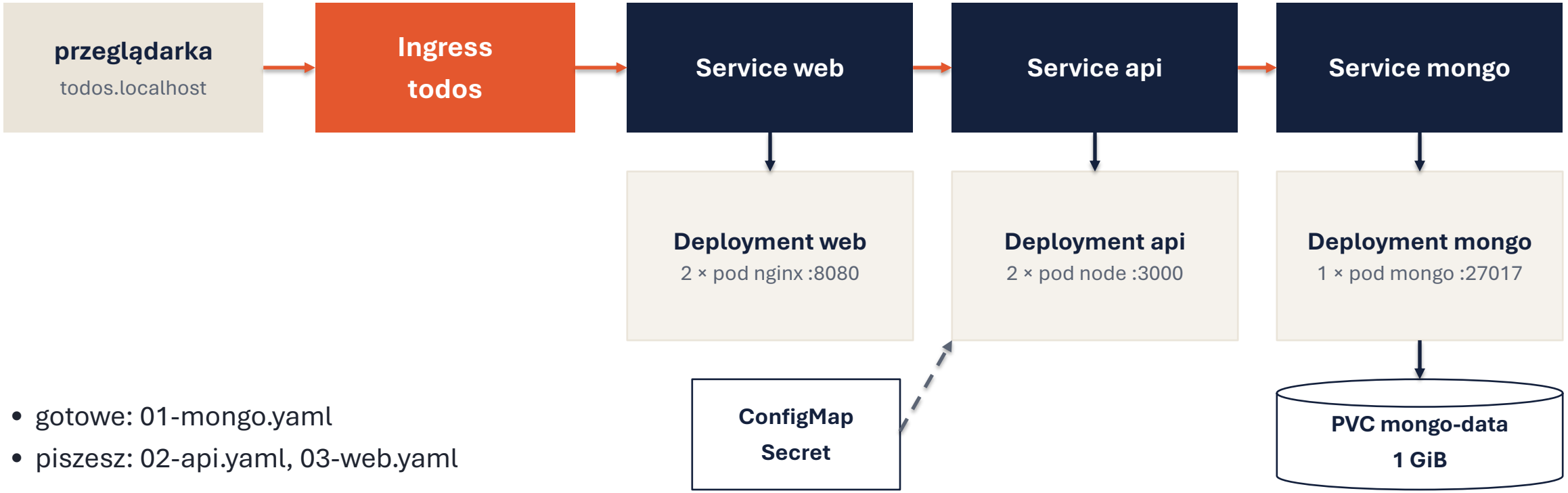
Po tym temacie Twoja aplikacja z wczoraj działa na Kubernetesie — z bazą, konfiguracją, sondami i adresem w przeglądarce.

1	2	3	4	5
Pod i Namespace	Deployment i konfiguracja	Service i Ingress	**ToDos na klastrze**	Co dalej

Z Compose na Kubernetesa: słownik

compose.yaml	Kubernetes
services.api (image, build)	Deployment api
nazwa usługi jako adres	Service api (ClusterIP)
environment:	ConfigMap + Secret → envFrom
healthcheck	readinessProbe / livenessProbe
volumes: mongo-data	PersistentVolumeClaim
ports: "8080:8080"	Service web + Ingress todos.localhost
--scale api=3	replicas: 3
depends_on	brak — aplikacja ponawia, readiness czeka

ToDos na klastrze: to, co zbudujesz w lab09



- gotowe: 01-mongo.yaml
- piszesz: 02-api.yaml, 03-web.yaml

Ten sam rysunek co wczoraj w Compose — każdy prostokąt to teraz obiekt Kubernetesa

LAB

`labs/lab09-todos-k8s/`

Ćwiczenie końcowe: ToDos na Kubernetesie

1. kind load: obrazy z wczoraj do klastra; apply 01-mongo.yaml
2. 02-api.yaml: ConfigMap, Secret, Deployment z sondami, Service api
3. 03-web.yaml: Deployment, Service web, Ingress todos.localhost
4. test: usuń pod bazy i pody API — dane i strona zostają

SUKCES: http://todos.localhost działa, zadania przeżywają kubectl delete pod -l app=mongo

Omówienie: co poszło nie tak — i czego to uczy

- ImagePullBackOff: skąd klaster bierze obrazy (rejestr, kind load)
- 0/1 READY: readiness odcina ruch, dopóki API nie ma bazy
- Brak połączenia z API: nazwa Service to adres w DNS
- 404 z Ingressu: host, Service i port muszą się zgadzać
- dane po usunięciu poda: PVC żyje niezależnie od poda

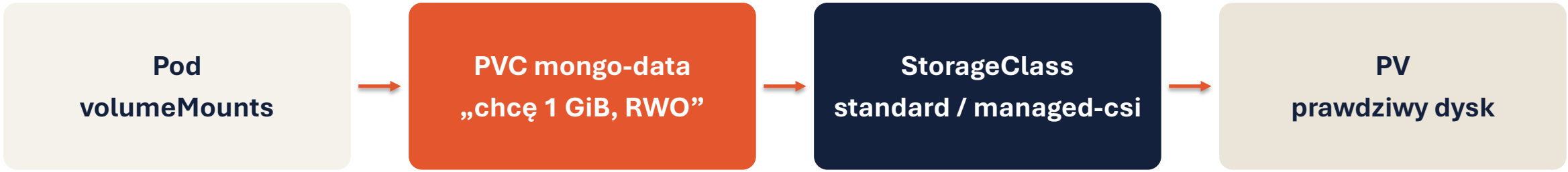
5

DZIEŃ 2 · TEMAT 5

Co dalej

Po tym temacie wiesz, czego jeszcze potrzeba, żeby aplikacja z labu była gotowa na produkcję — i gdzie się tego uczyć.

Dane: PersistentVolumeClaim prosi, StorageClass dostarcza



- PVC = zamówienie, PV = dysk; StorageClass tworzy dysk automatycznie
- kind: katalog na węźle · Azure: Azure Disk · AWS: EBS
- ReadWriteOnce: dysk podpięty do jednego węzła naraz (bazy danych)
- baza w produkcji: StatefulSet (stałe nazwy, dysk per replika) albo operator — lub baza zarządzana w chmurze

Nie tylko Deployment: kontrolery do innych zadań

Kontroler	Co robi	Przykład
Deployment	bezstanowe repliki, rolling update i rollback	api, web w ToDos
StatefulSet	stałe nazwy (mongo-0) i dysk na każdą replikę	bazy, kolejki, Kafka
DaemonSet	jeden pod na każdym węźle, także na nowym	agent logów, monitoring, kube-proxy
Job	uruchom do skutku i zakończ	migracja bazy, import danych
CronJob	Job według harmonogramu (składnia cron)	nocny backup, raporty

Każdy kontroler działa tak samo: opisujesz stan, a on go pilnuje

DEMO

Autoskalowanie: HPA dodaje pody pod obciążeniem

Horizontal Pod Autoscaler patrzy na CPU podów i zmienia replicas

```
cd demo/d2-04-hpa
kubectl apply -f app.yaml
kubectl apply -f hpa.yaml     # 1–5 replik, 50% CPU
kubectl get hpa -w            # drugi terminal

kubectl run obciazenie --rm -it --image=busybox:1.37 -- \
  sh -c 'while true; do wget -qO- http://liczydlo \
  >/dev/null; done'

kubectl top pods
```

OBSERWUJCIE

- TARGETS: zużycie CPU jako % requests
- REPLICAS rośnie do 5 w niecałą minutę
- po zatrzymaniu obciążenia spada po ~5 min
- bez requests HPA nie zadziała

Helm: menedżer pakietów dla Kubernetesa

```
helm repo add traefik https://traefik.github.io/charts
helm install traefik traefik/traefik \
  --version 42.3.1 \
  -n traefik --create-namespace \
  -f traefik-values.yaml

helm list -A
helm upgrade traefik traefik/traefik -f nowe.yaml
helm rollback traefik 1
helm uninstall traefik -n traefik
```

- chart = szablony YAML + values
- jeden chart, różne values: dev, prod
- wersje i rollback całej aplikacji
- gotowe charty: artifacthub.io
- tak instalowaliśmy Traefik w setup

Od labu do produkcji: czego jeszcze potrzeba

Bezpieczeństwo

- RBAC: kto co może w klastrze
- NetworkPolicy: kto z kim rozmawia
- Pod Security: bez roota, bez uprawnień
- skanowanie obrazów, sekrety w sejfie

Obserwowalność

- metryki: Prometheus + Grafana
- logi: Loki, Elastic, usługa chmury
- śledzenie: OpenTelemetry
- alerty, zanim zadzwoni klient

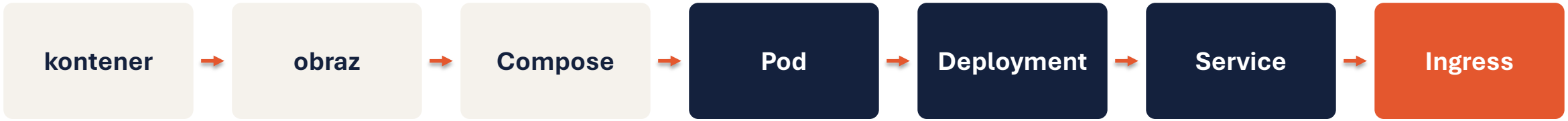
Dostarczanie

- CI/CD: build, test, skan, push
- GitOps: Argo CD, Flux
- Helm/Kustomize per środowisko
- strategie: canary, blue-green

Jak uczyć się dalej

- to repozytorium: laby „dla chętnych”, ściągi i diagramy z docs/ zostają z Wami
- kubernetes.io/docs/tutorials — oficjalne tutoriale
- klaster kind: zepsuj, napraw, usuń, utwórz od nowa — za darmo
- certyfikaty: KCNA (podstawy), CKAD (developer), CKA (administrator)
- manifesty z labów w Git + Helm chart dla ToDos — świetne ćwiczenie

Dwa dni: od kontenera do aplikacji na klastrze



- kontener to proces z izolacją — ulotny, dane i konfiguracja poza nim
- obraz budujesz mały, bez roota, z konkretną wersją
- w Kubernetesie opisujesz stan — kontrolery go pilnują
- Service daje stały adres, Ingress — adres dla przeglądarki
- diagnostyka: get → describe → logs

ToDos: od docker compose up do http://todos.localhost na klastrze

Dziękuję!

Pytania · ankieta Altkom · linkedin.com/in/dawid-marniok